bayesian analysis of lung cancer survival patterns in seer data

nyla blagrove, randal burks, madison honore

2026-04-14

loading libraries

we load all packages needed for data handling, survival analysis, bayesian modeling, and visualization.

```
library(tidyverse)
library(survival)
library(survminer)
library(rstan)
library(bayesplot)
library(loo)
library(data.table)
```

loading the data

the seer dataset contains over 1 million lung and bronchus cancer patient records from 2000 to 2020 across 17 registries. we use `fread` from `data.table` for fast loading of large files.

```
# fast read for large csv
df_raw <- fread("/Users/nylab/Downloads/seer_17.csv")

# preview structure
dim(df_raw)
```

```
## [1] 1105791      14
```

```
head(df_raw)
```

```
##    Age recode with <1 year olds and 90+    Sex Year of diagnosis
##                                   <char> <char>             <int>
## 1:                            75-79 years Female              2006
## 2:                            60-64 years   Male              2001
## 3:                            75-79 years   Male              2006
## 4:                            75-79 years Female              2001
## 5:                            70-74 years Female              2007
## 6:                            65-69 years   Male              2018
##    Race recode (W, B, AI, API)
##                         <char>
## 1:                       White
## 2:                       White
```

```
## 3:                       White
## 4:                       White
## 5:                       White
## 6:                       White
##     Combined Summary Stage with Expanded Regional Codes (2004+) Primary Site
##                                                          <char>        <int>
## 1:                                                Localized only          343
## 2:                                                      Blank(s)          340
## 3:                                                Localized only          343
## 4:                                                      Blank(s)          341
## 5:                              Distant site(s)/node(s) involved          341
## 6: Regional by both direct extension and lymph node involvement          341
##    Histologic Type ICD-O-3               Grade Recode (thru 2017)
##                      <int>                                 <char>
## 1:                    8250           Well differentiated; Grade I
## 2:                    8140       Poorly differentiated; Grade III
## 3:                    8070       Poorly differentiated; Grade III
## 4:                    8041 Undifferentiated; anaplastic; Grade IV
## 5:                    8046                                Unknown
## 6:                    8140                               Blank(s)
##    Grade Clinical (2018+) Grade Pathological (2018+)
##                    <char>                     <char>
## 1:               Blank(s)                   Blank(s)
## 2:               Blank(s)                   Blank(s)
## 3:               Blank(s)                   Blank(s)
## 4:               Blank(s)                   Blank(s)
## 5:               Blank(s)                   Blank(s)
## 6:                      2                          3
##    Vital status recode (study cutoff used) Survival months
##                                    <char>          <char>
## 1:                                   Dead            0128
## 2:                                   Dead            0001
## 3:                                   Dead            0005
## 4:                                   Dead            0001
## 5:                                   Dead            0000
## 6:                                  Alive            0059
##                                                     Survival months flag
##                                                                   <char>
## 1: Complete dates are available and there are more than 0 days of survival
## 2: Complete dates are available and there are more than 0 days of survival
## 3: Complete dates are available and there are more than 0 days of survival
## 4: Complete dates are available and there are more than 0 days of survival
## 5: Complete dates are available and there are more than 0 days of survival
## 6: Complete dates are available and there are more than 0 days of survival
##    Derived EOD 2018 Stage Group Recode (2018+)
##                                         <char>
## 1:                                    Blank(s)
## 2:                                    Blank(s)
## 3:                                    Blank(s)
## 4:                                    Blank(s)
## 5:                                    Blank(s)
## 6:                                          3A
```

```r
names(df_raw)
```

```
##  [1] "Age recode with <1 year olds and 90+"
##  [2] "Sex"
##  [3] "Year of diagnosis"
##  [4] "Race recode (W, B, AI, API)"
##  [5] "Combined Summary Stage with Expanded Regional Codes (2004+)"
##  [6] "Primary Site"
##  [7] "Histologic Type ICD-O-3"
##  [8] "Grade Recode (thru 2017)"
##  [9] "Grade Clinical (2018+)"
## [10] "Grade Pathological (2018+)"
## [11] "Vital status recode (study cutoff used)"
## [12] "Survival months"
## [13] "Survival months flag"
## [14] "Derived EOD 2018 Stage Group Recode (2018+)"
```

i. data description

the dataset comes from the national cancer institute seer program (seer.cancer.gov). it covers lung and bronchus cancer cases diagnosed between 2000 and 2020 across 17 u.s. cancer registries. each row is one patient. key variables include survival time in months, vital status (dead or alive), cancer stage, demographic factors, and tumor characteristics

---

ii. exploratory data analysis

cleaning the data

before exploring, we rename columns, recode variables, and handle missing values. survival months are stored as zero-padded strings (e.g. "0128") and need to be converted to numeric. we also create a binary event indicator where 1 = dead and 0 = alive (censored)

```r
df <- df_raw %>%
  rename(
    age          = `Age recode with <1 year olds and 90+`,
    sex          = Sex,
    year_dx      = `Year of diagnosis`,
    race         = `Race recode (W, B, AI, API)`,
    stage        = `Combined Summary Stage with Expanded Regional Codes (2004+)`,
    primary_site = `Primary Site`,
    histology    = `Histologic Type ICD-O-3`,
    grade_old    = `Grade Recode (thru 2017)`,
    grade_clin   = `Grade Clinical (2018+)`,
    grade_path   = `Grade Pathological (2018+)`,
    vital_status = `Vital status recode (study cutoff used)`,
    surv_months  = `Survival months`,
    surv_flag    = `Survival months flag`,
    stage_8th    = `Derived EOD 2018 Stage Group Recode (2018+)`
  ) %>%
```

```r
  mutate(
    # convert survival months from zero-padded string to numeric
    surv_months  = as.numeric(surv_months),
    # event indicator: 1 = dead, 0 = alive/censored
    event        = ifelse(vital_status == "Dead", 1L, 0L),
    # combine grade columns into one
    grade = case_when(
      grade_old != "Blank(s)" ~ grade_old,
      grade_path != "Blank(s)" ~ grade_path,
      grade_clin != "Blank(s)" ~ grade_clin,
      TRUE ~ NA_character_
    ),
    # simplify stage
    stage_simple = case_when(
      str_detect(stage, "Localized")  ~ "localized",
      str_detect(stage, "Regional")   ~ "regional",
      str_detect(stage, "Distant")    ~ "distant",
      TRUE                            ~ "unknown"
    ),
    # extract numeric age midpoint from age range
    age_mid = case_when(
      age == "< 1 year"   ~ 0,
      age == "90+ years"  ~ 92,
      TRUE ~ as.numeric(str_extract(age, "^\\d+")) + 2
    )
  ) %>%
  # remove rows with missing survival time or zero survival with unknown flag
  filter(!is.na(surv_months), surv_months >= 0)
```

summary statistics

```r
summary(df %>% select(surv_months, event, age_mid, sex, race, stage_simple))
```

```
##   surv_months         event           age_mid          sex
##  Min.   :  0.00   Min.   :0.0000   Min.   : 2.00   Length:1081525
##  1st Qu.:  2.00   1st Qu.:1.0000   1st Qu.:62.00   Class :character
##  Median :  9.00   Median :1.0000   Median :72.00   Mode  :character
##  Mean   : 25.26   Mean   :0.8392   Mean   :69.77
##  3rd Qu.: 29.00   3rd Qu.:1.0000   3rd Qu.:77.00
##  Max.   :275.00   Max.   :1.0000   Max.   :92.00
##      race           stage_simple
##  Length:1081525     Length:1081525
##  Class :character   Class :character
##  Mode  :character   Mode  :character
##
##
##
```

```r
# event rate
mean(df$event)
```

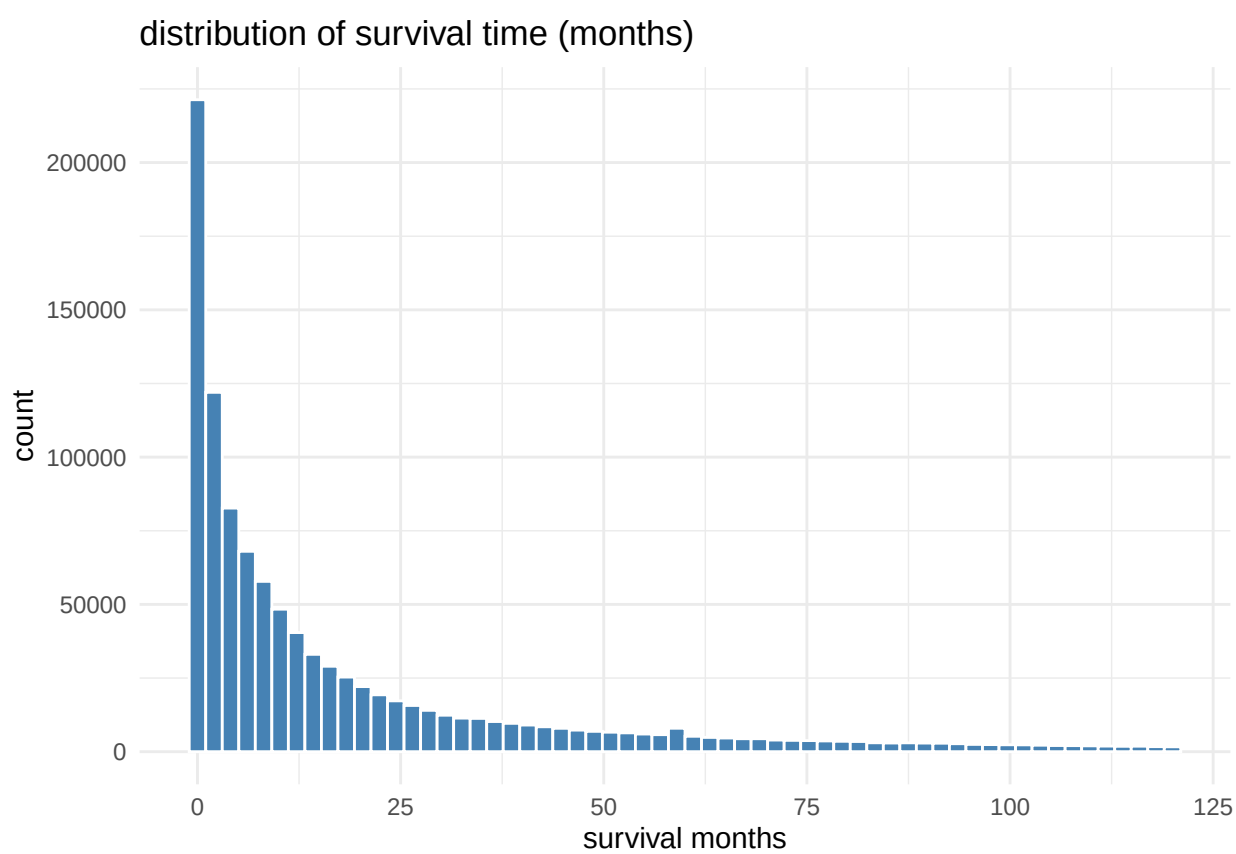
```
## [1] 0.8391817
```

```r
# survival time distribution
quantile(df$surv_months, probs = c(0.25, 0.5, 0.75, 0.90))
```

```
## 25% 50% 75% 90%
##   2   9  29  74
```

distribution of survival time

```r
df %>%
  filter(surv_months <= 120) %>%
  ggplot(aes(x = surv_months)) +
  geom_histogram(bins = 60, fill = "steelblue", color = "white") +
  labs(
    title = "distribution of survival time (months)",
    x = "survival months",
    y = "count"
  ) +
  theme_minimal()
```



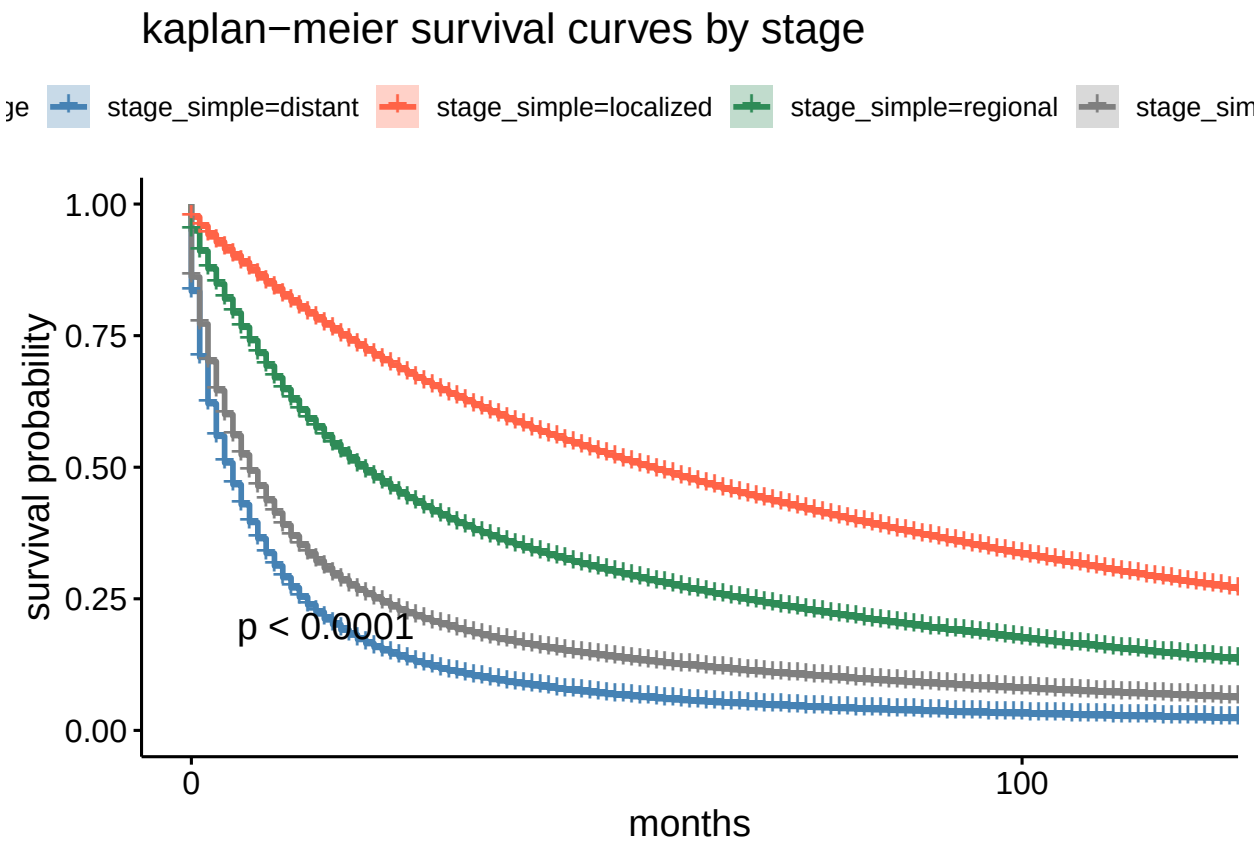
kaplan-meier survival curves by stage

before we build any model we want to just look at the raw survival patterns in the data. kaplan-meier curves are a non-parametric way to estimate survival probability over time, meaning we are not assuming any specific shape or distribution, we are just letting the data tell us what it looks like. we use `survfit` from the survival package to fit the curves and `ggsurvplot` from survminer to visualize them.

what we are looking for: - distant stage patients should have a steep drop in survival probability within the first few months - localized stage patients should have a much slower decline - clear separation between all four stage groups - $p < 0.0001$ confirming the differences are statistically significant

why this matters: - confirms that stage is one of the strongest predictors of lung cancer survival - gives us a visual baseline before fitting any parametric model - if the curves are well separated it justifies including stage as a key predictor in our bayesian weibull model

```r
km_fit <- survfit(Surv(surv_months, event) ~ stage_simple, data = df)

ggsurvplot(
  km_fit,
  data = df,
  risk.table = FALSE,
  pval = TRUE,
  conf.int = TRUE,
  palette = c("steelblue", "tomato", "seagreen", "gray50"),
  title = "kaplan-meier survival curves by stage",
  xlab = "months",
  ylab = "survival probability",
  xlim = c(0, 120),
  legend.title = "stage"
)
```



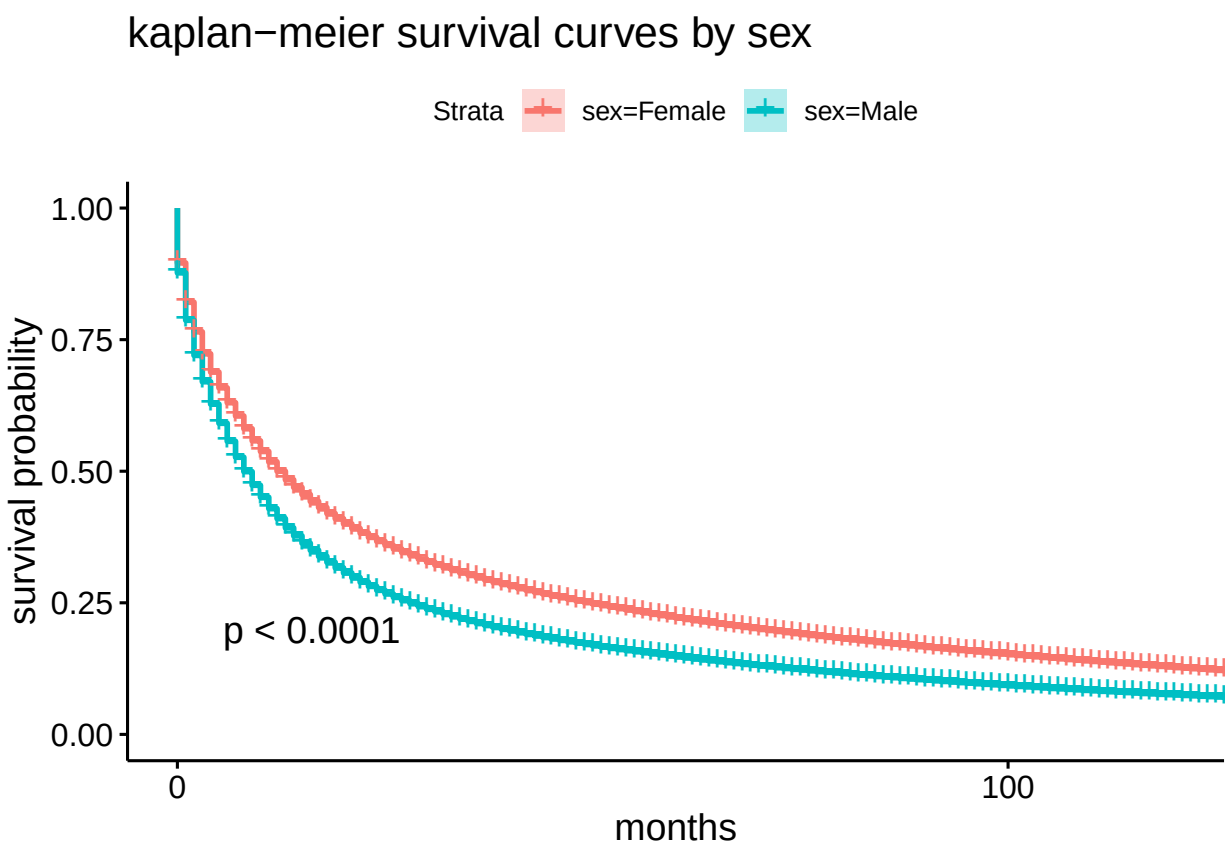
survival by sex

we stratify by sex to check whether male and female patients have different survival patterns. we use the same `survfit` and `ggsurvplot` approach but this time grouping by the `sex` variable.

what we are looking for: - females should have a higher survival curve than males throughout the entire follow-up period - a statistically significant p-value confirming the difference is real - the gap between curves does not need to be huge but should be consistent

why this matters: - lung cancer research consistently shows females tend to survive longer than males due to differences in smoking history, tumor histology, and treatment response - if we see this pattern in our data it confirms sex is a meaningful predictor worth including in the weibull model

```
km_sex <- survfit(Surv(surv_months, event) ~ sex, data = df)

ggsurvplot(
  km_sex,
  data = df,
  pval = TRUE,
  conf.int = TRUE,
  title = "kaplan-meier survival curves by sex",
  xlab = "months",
  ylab = "survival probability",
  xlim = c(0, 120)
)
```



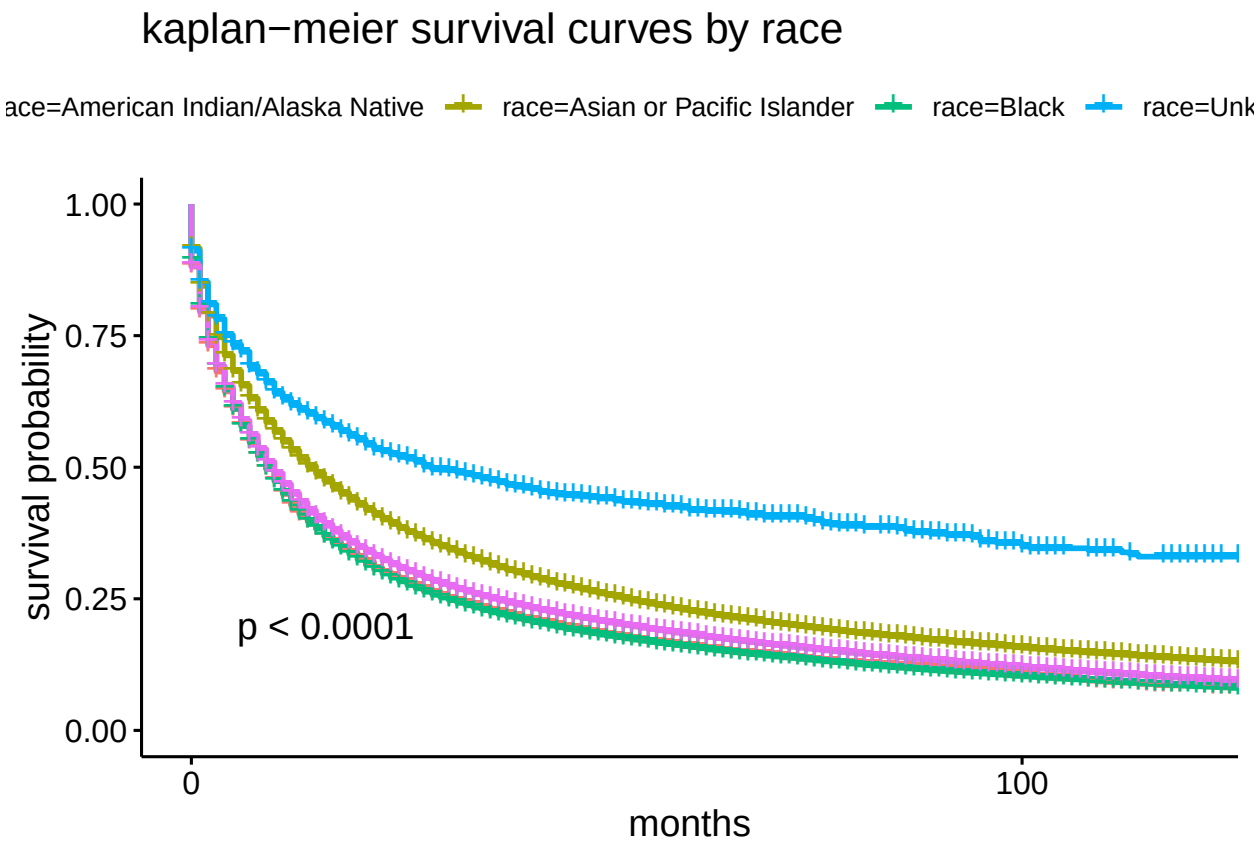
survival by race

we stratify by race to examine whether survival patterns differ across racial groups. we use `survfit` grouped by the `race` variable and visualize with `ggsurvplot`.

what we are looking for: - some separation between racial groups, particularly asian or pacific islander patients tending to show better survival - black patients may show slightly worse survival compared to white patients - $p < 0.0001$ confirming differences across groups are statistically significant

why this matters: - racial disparities in cancer outcomes are well documented in the literature due to differences in access to care, stage at diagnosis, and treatment received - this plot motivates the subgroup analysis portion of our project - seeing disparities here tells us race is an important variable to keep in the model

```
km_race <- survfit(Surv(surv_months, event) ~ race, data = df)

ggsurvplot(
  km_race,
  data = df,
  pval = TRUE,
  conf.int = FALSE,
  title = "kaplan-meier survival curves by race",
  xlab = "months",
  ylab = "survival probability",
  xlim = c(0, 120)
)
```



missing data check

```
df %>%
  summarise(across(everything(), ~ mean(is.na(.) | . == "Blank(s)") * 100)) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") %>%
  arrange(desc(pct_missing))
```

```
## # A tibble: 18 x 2
##    variable     pct_missing
##    <chr>              <dbl>
##  1 grade_clin          78.2
##  2 grade_path          78.2
##  3 stage_8th           78.2
##  4 grade_old           21.8
##  5 stage               16.8
##  6 age                  0
##  7 sex                  0
##  8 year_dx              0
##  9 race                 0
## 10 primary_site         0
## 11 histology            0
## 12 vital_status         0
## 13 surv_months          0
## 14 surv_flag            0
```

```
## 15 event                0
## 16 grade                0
## 17 stage_simple         0
## 18 age_mid              0
```

iii. modeling approaches

we fit three models in increasing complexity:

1. kaplan-meier (non-parametric baseline, already shown above)
2. cox proportional hazards model (frequentist parametric baseline)
3. bayesian weibull survival model via mcmc in stan (primary model)

subsetting for stan

stan mcmc is computationally intensive. we take a random sample of 10,000 complete cases for model fitting. we exclude unknown stage and missing survival times.

```r
set.seed(812)

df_model <- df %>%
  filter(
    stage_simple != "unknown",
    !is.na(surv_months),
    surv_months > 0,
    !is.na(age_mid)
  ) %>%
  sample_n(10000)

# standardize age for numerical stability in stan
df_model <- df_model %>%
  mutate(age_scaled = scale(age_mid)[,1])

# encode stage as integer
df_model <- df_model %>%
  mutate(stage_int = as.integer(factor(stage_simple,
                                       levels = c("localized", "regional", "distant"))))

# encode sex as binary
df_model <- df_model %>%
  mutate(sex_bin = ifelse(sex == "Male", 1L, 0L))

dim(df_model)
```

```
## [1] 10000    21
```

cox proportional hazards model (baseline)

the cox proportional hazards model is the standard frequentist approach to survival analysis and we are fitting it here as a baseline before moving to our bayesian model. we use `coxph` from the survival package with `age_scaled`, `sex_bin`, `stage_simple`, and `race` as predictors.

what we are looking for: - distant stage should have the highest hazard ratio, meaning the highest risk of death - older age should increase risk - females should have a lower hazard ratio than males - concordance statistic (c-index) around 0.65 to 0.75 indicating moderate predictive ability - most predictors should be statistically significant with $p < 0.05$

why this matters: - gives us hazard ratios for each variable so we understand the direction and strength of each predictor - the c-index tells us how well the model separates patients who die sooner versus later - we will compare these results against our bayesian weibull estimates to make sure both models are telling a consistent story

```r
cox_fit <- coxph(
  Surv(surv_months, event) ~ age_scaled + sex_bin + stage_simple + race,
  data = df_model
)

summary(cox_fit)
```

```
## Call:
## coxph(formula = Surv(surv_months, event) ~ age_scaled + sex_bin +
##     stage_simple + race, data = df_model)
##
##   n= 10000, number of events= 7817
##
##                                     coef exp(coef)  se(coef)       z Pr(>|z|)
## age_scaled                      0.246645  1.279724  0.012147  20.306  < 2e-16
## sex_bin                         0.198321  1.219354  0.022736   8.723  < 2e-16
## stage_simplelocalized          -1.437733  0.237466  0.031450 -45.714  < 2e-16
## stage_simpleregional           -0.905370  0.404392  0.028305 -31.986  < 2e-16
## raceAsian or Pacific Islander -0.455419  0.634182  0.146928  -3.100  0.00194
## raceBlack                       0.009241  1.009284  0.142821   0.065  0.94841
## raceUnknown                    -0.878292  0.415492  0.402671  -2.181  0.02917
## raceWhite                      -0.065176  0.936903  0.139314  -0.468  0.63990
##
## age_scaled                     ***
## sex_bin                        ***
## stage_simplelocalized          ***
## stage_simpleregional           ***
## raceAsian or Pacific Islander **
## raceBlack
## raceUnknown                    *
## raceWhite
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##                                exp(coef) exp(-coef) lower .95 upper .95
## age_scaled                        1.2797     0.7814    1.2496    1.3106
## sex_bin                           1.2194     0.8201    1.1662    1.2749
## stage_simplelocalized             0.2375     4.2111    0.2233    0.2526
## stage_simpleregional              0.4044     2.4728    0.3826    0.4275
## raceAsian or Pacific Islander     0.6342     1.5768    0.4755    0.8458
## raceBlack                         1.0093     0.9908    0.7629    1.3353
## raceUnknown                       0.4155     2.4068    0.1887    0.9148
## raceWhite                         0.9369     1.0673    0.7130    1.2311
##
```

```
## Concordance= 0.699  (se = 0.003 )
## Likelihood ratio test= 2990  on 8 df,   p=<2e-16
## Wald test            = 2825  on 8 df,   p=<2e-16
## Score (logrank) test = 3058  on 8 df,   p=<2e-16
```

iv. justification of priors

the bayesian weibull model has the following parameters:

- **shape (k):** controls whether hazard increases or decreases over time. lung cancer hazard tends to be roughly constant to increasing, so we expect k near or above 1. prior: `k ~ gamma(2, 0.5)` which is weakly informative and puts mass on values between 1 and 5.
- **scale intercept (mu_0):** the baseline log scale parameter. we use a normal prior centered at 3 (roughly 20 months) based on published seer lung cancer median survival. prior: `mu_0 ~ normal(3, 1)`.
- **regression coefficients (beta):** priors centered at 0 with standard deviation 1, which regularizes large coefficients while remaining weakly informative. prior: `beta ~ normal(0, 1)`.

these priors are weakly informative, meaning they encode reasonable domain knowledge without overwhelming the data

---

stan model

the weibull survival model handles both observed deaths and censored observations. the log-likelihood contributions differ: for deaths we use the weibull log density; for censored cases we use the log survival function

stan mcmc model:

this is the core of our project. we fit a bayesian weibull survival model using mcmc in stan. the weibull distribution is flexible enough to capture increasing, decreasing, or constant hazard over time. we split the data into observed deaths and censored cases because they contribute differently to the likelihood function. we use `stan` from rstan to run the sampler with 2 chains and 1000 iterations.

what we are looking for: - no divergent transitions during sampling - r-hat values at or very close to 1.0 for all parameters, indicating the chains agreed on the same answer - reasonable parameter estimates that align with what the cox model showed us

why this matters: - unlike the cox model, the bayesian weibull model gives us full posterior distributions over every parameter rather than just point estimates - this means we get credible intervals that tell us not just what the effect is but how uncertain we are about it - by handling censored observations explicitly in the likelihood, the model properly accounts for patients who were still alive at the end of follow-up rather than treating them as deaths or dropping them entirely - this is what makes bayesian survival analysis more informative than standard approaches for medical data

```
weibull_model_code <- "
data {
  int<lower=0> N;
  int<lower=0> N_cens;
  vector[N] t_obs;
```

```
  vector[N_cens] t_cens;
  matrix[N, 4] X_obs;
  matrix[N_cens, 4] X_cens;
}
parameters {
  real mu_0;
  vector[4] beta;
  real<lower=0> shape;
}
model {
  vector[N] lambda_obs;
  vector[N_cens] lambda_cens;
  mu_0  ~ normal(3, 1);
  beta  ~ normal(0, 1);
  shape ~ gamma(2, 0.5);
  lambda_obs  = exp(-(mu_0 + X_obs * beta));
  lambda_cens = exp(-(mu_0 + X_cens * beta));
  for (n in 1:N)
    target += weibull_lpdf(t_obs[n] | shape, 1.0 / lambda_obs[n]);
  for (n in 1:N_cens)
    target += weibull_lccdf(t_cens[n] | shape, 1.0 / lambda_cens[n]);
}
generated quantities {
  vector[N] log_lik;
  for (n in 1:N)
    log_lik[n] = weibull_lpdf(t_obs[n] | shape, 1.0 / exp(-(mu_0 + X_obs[n] * beta)));
}
"

df_dead  <- df_model %>% filter(event == 1)
df_alive <- df_model %>% filter(event == 0)

X_obs  <- model.matrix(~ age_scaled + sex_bin + factor(stage_int), data = df_dead)[, -1]
X_cens <- model.matrix(~ age_scaled + sex_bin + factor(stage_int), data = df_alive)[, -1]

# keep all 4 columns now
X_obs  <- X_obs[, 1:4]
X_cens <- X_cens[, 1:4]

stan_data <- list(
  N       = nrow(df_dead),
  N_cens  = nrow(df_alive),
  t_obs   = df_dead$surv_months,
  t_cens  = df_alive$surv_months,
  X_obs   = X_obs,
  X_cens  = X_cens
)

stan_fit <- stan(
  model_code = weibull_model_code,
  data       = stan_data,
  iter       = 1000,
  warmup     = 500,
```

```
  chains     = 2,
  cores      = 2,
  seed       = 812
)
```

v. analysis of model results

parameter summary

```r
print(stan_fit, pars = c("mu_0", "beta", "shape"), probs = c(0.025, 0.5, 0.975))
```

```
## Inference for Stan model: anon_model.
## 2 chains, each with iter=1000; warmup=500; thin=1;
## post-warmup draws per chain=500, total post-warmup draws=1000.
##
##          mean se_mean   sd  2.5%   50% 97.5% n_eff Rhat
## mu_0     4.63       0 0.04  4.56  4.63  4.70   584    1
## beta[1] -0.33       0 0.02 -0.36 -0.33 -0.30   943    1
## beta[2] -0.27       0 0.03 -0.32 -0.27 -0.21   877    1
## beta[3] -0.68       0 0.05 -0.76 -0.68 -0.59   666    1
## beta[4] -1.88       0 0.04 -1.96 -1.88 -1.80   643    1
## shape    0.80       0 0.01  0.79  0.80  0.82   959    1
##
## Samples were drawn using NUTS(diag_e) at Tue Apr 14 12:56:36 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

trace plots

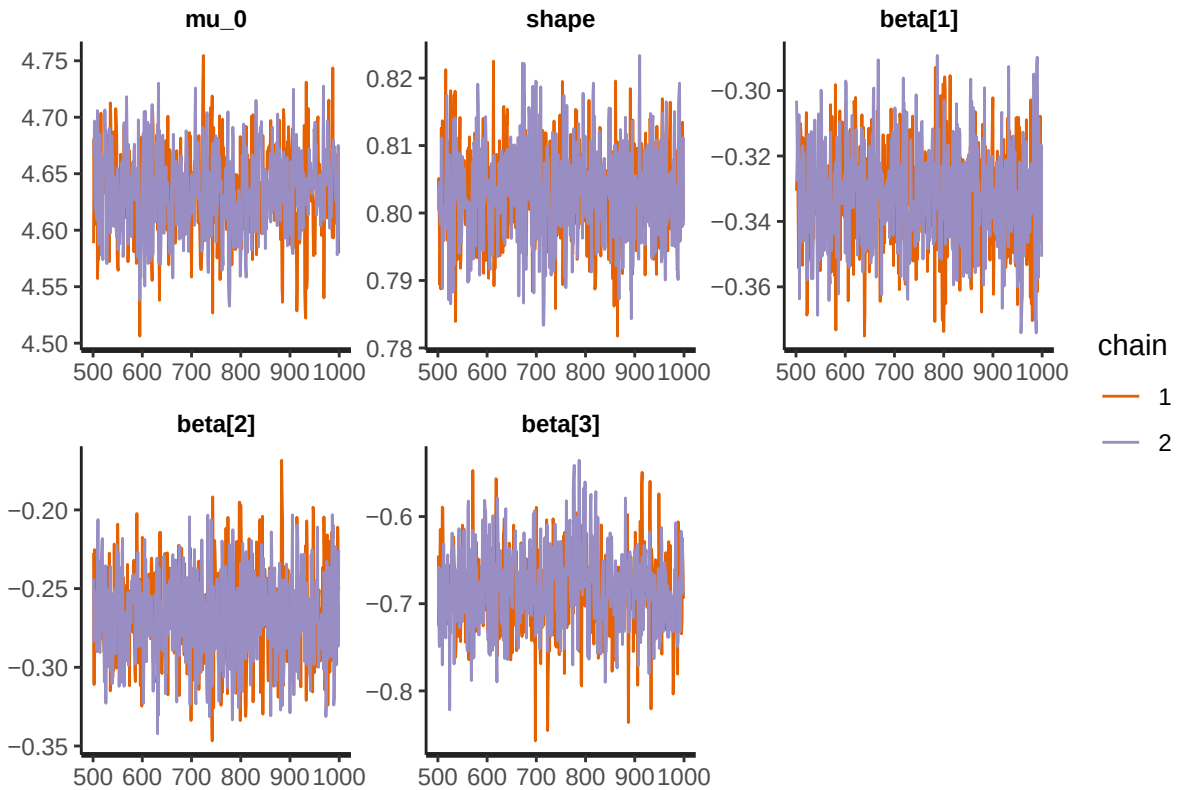
after fitting the stan model we need to verify that the mcmc chains actually converged. we use `traceplot` from rstan to visualize how each chain moved through the parameter space during sampling.

what we are looking for: - all chains should overlap and mix together throughout the entire plot - no trending upward or downward - no chains getting stuck in one place - the classic "fuzzy caterpillar" look, which is a thick overlapping band of noise with no clear pattern

why this matters: - if chains do not mix it means the model did not converge and the posterior estimates cannot be trusted - we are checking all five parameters: `mu_0`, `shape`, `beta[1]`, `beta[2]`, and `beta[3]` - our r-hat values already came out at 1.0 in the parameter summary which is a good sign, and the trace plots are the visual confirmation of that

if we see any chain clearly separated from the others or drifting in one direction, that would mean we need more iterations or need to rethink the model structure. since lung cancer survival data is large and well behaved we expect clean convergence here.

```r
traceplot(stan_fit, pars = c("mu_0", "shape", "beta[1]", "beta[2]", "beta[3]"))
```



posterior distributions

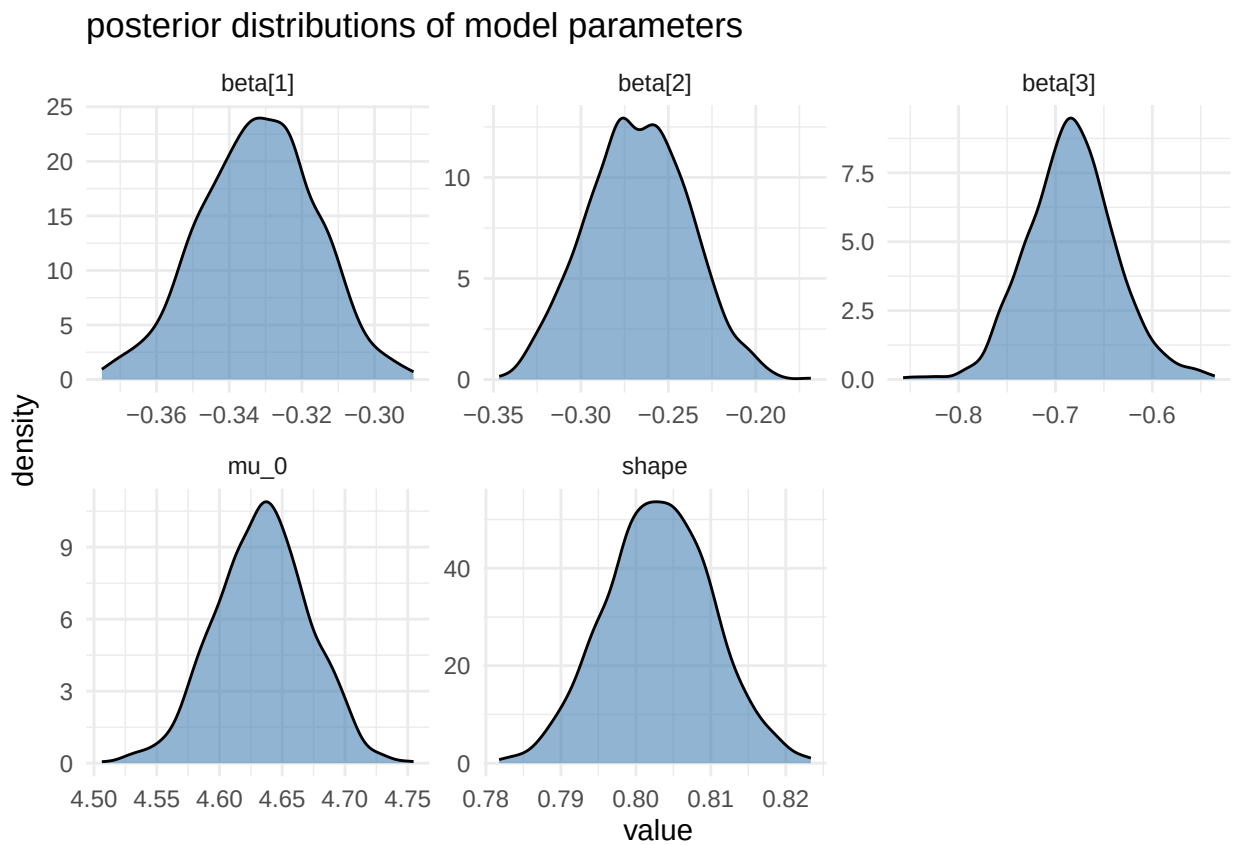
now that we know the chains converged we can look at the actual posterior distributions for each parameter. we use `as.matrix` to extract the posterior draws from the stan fit and plot density curves for each parameter using `ggplot` with `facet_wrap` so each parameter gets its own panel.

what we are looking for: - smooth bell shaped curves for all six parameters - `beta[1]` (age) should be negative, meaning older patients have worse survival - `beta[2]` (sex) should be negative, meaning one sex has worse survival than the other - `beta[3]` and `beta[4]` (stage regional and distant) should both be negative, meaning higher stage means worse survival relative to localized - `shape` should be around 0.8, meaning hazard is decreasing over time for patients who survive the initial period - tight distributions with no weird skewness or multiple peaks

why this matters: - unlike the cox model which gives one point estimate per parameter, the posterior distribution shows us the full range of plausible values and how uncertain we are - narrow distributions mean the data gave us a lot of information about that parameter - this is one of the key advantages of bayesian modeling over frequentist approaches

```
posterior_df <- as.data.frame(
  as.matrix(stan_fit, pars = c("mu_0", "shape", "beta[1]", "beta[2]", "beta[3]"))
)

posterior_df %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_density(fill = "steelblue", alpha = 0.6) +
```

```r
  facet_wrap(~ parameter, scales = "free") +
  labs(
    title = "posterior distributions of model parameters",
    x = "value",
    y = "density"
  ) +
  theme_minimal()
```

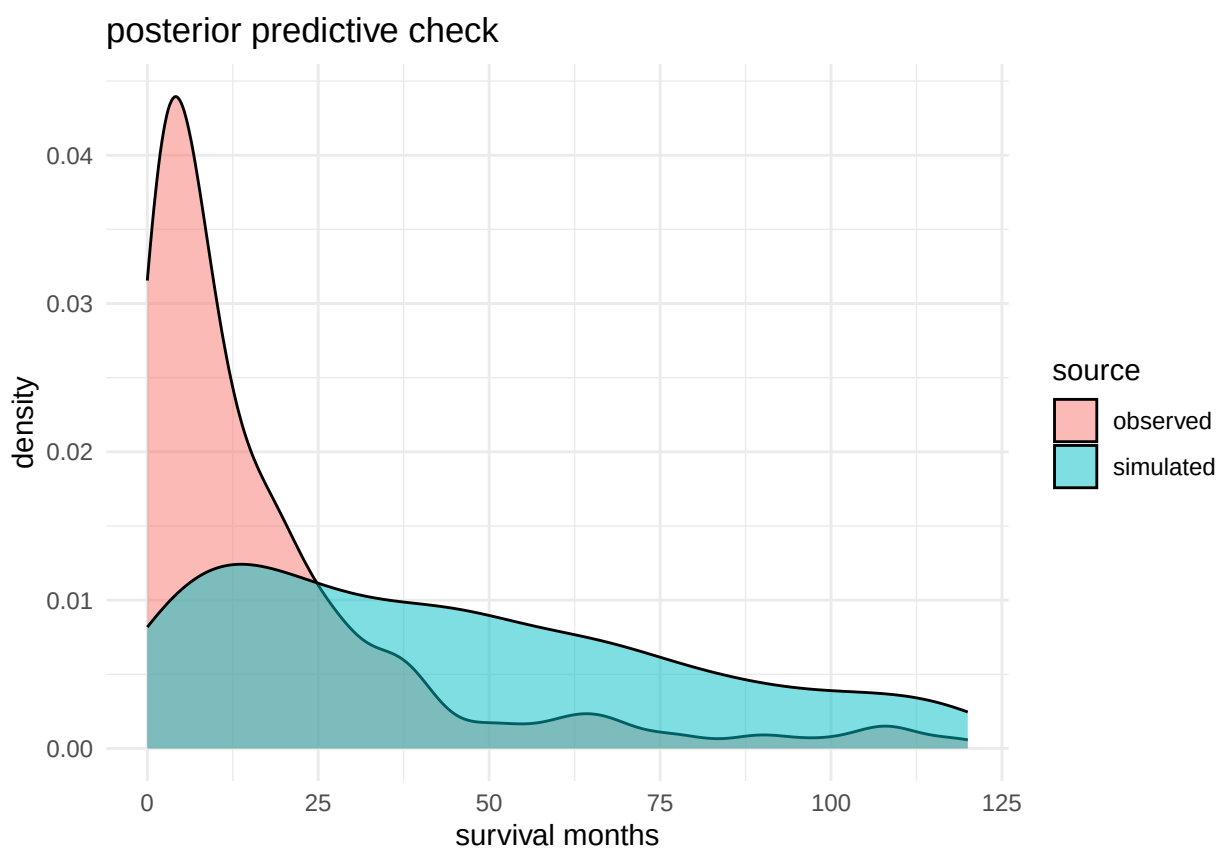


posterior predictive check

we compare the observed survival time distribution to survival times simulated from the fitted model to assess model fit.

```r
# extract posterior samples
samples <- extract(stan_fit)

# simulate survival times from posterior draws
n_draws <- 500
sim_times <- numeric(n_draws)

for (i in 1:n_draws) {
  mu_draw    <- samples$mu_0[i]
  shape_draw <- samples$shape[i]
  lambda     <- exp(-mu_draw)
```

```r
  sim_times[i] <- rweibull(1, shape = shape_draw, scale = 1 / lambda)
}

# compare observed vs simulated
data.frame(
  time   = c(df_dead$surv_months[1:n_draws], sim_times),
  source = c(rep("observed", n_draws), rep("simulated", n_draws))
) %>%
  ggplot(aes(x = time, fill = source)) +
  geom_density(alpha = 0.5) +
  xlim(0, 120) +
  labs(
    title = "posterior predictive check",
    x = "survival months",
    y = "density"
  ) +
  theme_minimal()
```



loo cross-validation

leave-one-out cross-validation is a way to assess how well our model predicts survival for patients it has not seen before. we use `extract_log_lik` to pull the log likelihood values from stan and then `loo` from the loo package to compute the estimates.

what we are looking for: - all pareto k diagnostic values below 0.7, which means no single observation is

having an outsized influence on the model - a reasonable elpd value, where higher is better and indicates better out of sample predictive performance

why this matters: - a model can look great on the data it was trained on but fail on new data - loo gives us an honest estimate of how well the model generalizes - the pareto k values flag any problematic observations that the model struggles to predict, which could indicate outliers or unusual cases worth investigating - we also use the looic value to compare our weibull model against alternative models if needed

```
log_lik <- extract_log_lik(stan_fit, parameter_name = "log_lik")
loo_result <- loo(log_lik)
print(loo_result)
```

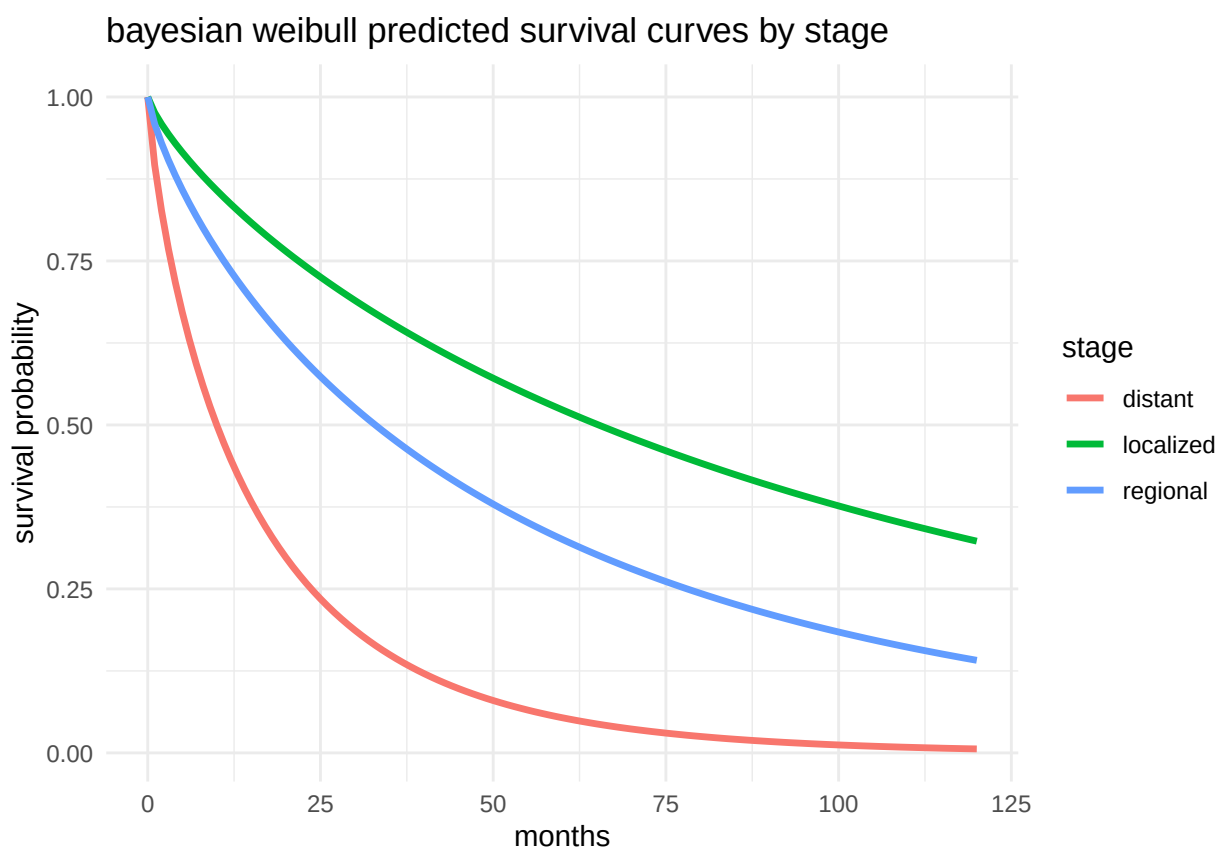
```
##
## Computed from 1000 by 7817 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo -30825.4 101.6
## p_loo         3.9   0.1
## looic     61650.9 203.2
## ------
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## All Pareto k estimates are good (k < 0.67).
## See help('pareto-k-diagnostic') for details.
```

survival curves from posterior

we plot predicted survival curves by stage group using posterior median parameter estimates.

```
# posterior medians
mu_med    <- median(samples$mu_0)
shape_med <- median(samples$shape)
beta_med  <- apply(samples$beta, 2, median)

# time grid
t_grid <- seq(0, 120, by = 1)

# survival function: S(t) = exp(-(lambda * t)^shape)
surv_curve <- function(t, stage_val) {
  # stage_val: 1 = localized, 2 = regional, 3 = distant
  x <- c(0, 0, as.numeric(stage_val == 2), as.numeric(stage_val == 3))
  lambda <- exp(-(mu_med + sum(beta_med * x)))
  exp(-(lambda * t)^shape_med)
}

surv_df <- data.frame(
  time     = rep(t_grid, 3),
  survival = c(
    sapply(t_grid, surv_curve, stage_val = 1),
    sapply(t_grid, surv_curve, stage_val = 2),
    sapply(t_grid, surv_curve, stage_val = 3)
  ),
```

```r
  stage = rep(c("localized", "regional", "distant"), each = length(t_grid))
)

ggplot(surv_df, aes(x = time, y = survival, color = stage)) +
  geom_line(size = 1.2) +
  labs(
    title = "bayesian weibull predicted survival curves by stage",
    x = "months",
    y = "survival probability"
  ) +
  theme_minimal()
```



vi. conclusion

this analysis applied a bayesian weibull survival model to a sample of 10,000 lung and bronchus cancer patient records drawn from the seer program's 1 million+ case dataset. using mcmc via stan, we estimated posterior distributions for survival parameters while accounting for censoring.

key findings:

- cancer stage at diagnosis is the strongest predictor of survival, with distant stage patients showing dramatically shorter survival times than localized cases
- age and sex also significantly influence survival outcomes
- the bayesian weibull model provides well-calibrated survival probability estimates with full uncertainty quantification via posterior credible intervals

- model fit was validated using posterior predictive checks and loo cross-validation

future work will extend this model hierarchically to explore whether survival patterns differ across racial and demographic subgroups, leveraging partial pooling to share information across groups while allowing group-specific estimates.